AI/ML 15-Month Curriculum

DAYS 1 – 10 ✦ CRAZY PROJECT

Smart Life Dashboard

A terminal-based personal intelligence system that tracks your habits, analyses your mood, predicts your productivity, and roasts you when you slack — built entirely from Python fundamentals.

Topics Inside:

✦ Day 1: Variables & Data Types
✦ Day 2: print() & input()
✦ Day 3: Conditions (if / elif / else)
✦ Day 4: Loops (for / while)
✦ Day 5: Lists & Tuples
✦ Day 6: Dictionaries
✦ Day 7: Functions
✦ Day 8: *args, **kwargs & Lambda
✦ Day 9: Modules & Imports
✦ Day 10: File I/O

HINTS ONLY — Not a Solution Sheet

What You're Building

The Smart Life Dashboard is a command-line Python app a real person would actually USE every day. It greets you, asks about your morning, checks your habits, computes a productivity score, stores everything in a file, loads your history, shows streaks and stats, and either praises you or roasts you. Every single feature uses one of Days 1–10. No external libraries. Pure Python fundamentals.

FEATURE	WHAT IT DOES	DAYS
Greeting system	Asks name, time of day, personalises welcome	`D1, D2, D3`
Mood tracker	Asks 1-10 score + mood word, stores it	`D2, D5, D6`
Habit check-in	Loops your habits list, records done/skip	`D4, D5`
Score calculator	Computes productivity from mood + habits + sleep	`D7, D8`
Roast/praise engine	Conditional messages based on score	`D3, D5, D7`
History logger	Appends today's entry to JSON file	`D10`
Stats viewer	Loads file, loops history, prints averages	`D4, D7, D10`
Streak counter	Counts consecutive good days	`D4, D6, D7`
Prediction hint	Naive tomorrow estimate from last 3 days	`D7, D8`

One file: dashboard.py. One data file: history.json. No folders, no classes, no OOP — just functions, loops, and files.

DAY 1	Variables & Data Types

Role in the project: The Foundation — every value your app stores starts as a variable.

Before the app can greet, compute, or save anything it must hold values in memory. Strings, ints, floats, booleans — the dashboard uses all four simultaneously in every session.

Hints for Your Implementation

1. User profile vars	Store name (str), daily goal (int e.g. 7), is_weekend (bool). Three lines, three types.
2. Session declarations	Each session needs: mood_score (int 1-10), habit_score (float 0-1), daily_score (float 0-10). Declare them at the top of your session function.
3. Constants block	Put thresholds at module level in ALL_CAPS: MIN_SCORE=5, STREAK_GOAL=7, HISTORY_FILE='history.json'. Day 1 variables meet engineering discipline.
4. Type conversion trap	input() always returns a string. You will need int() and float() everywhere. What if the user types 'great' instead of '7'? That's a Day 3 problem — but knowing the conversion is needed is a Day 1 insight.

Starter Code Snippet (hint — not a solution)

```
APP_NAME  = 'Smart Life Dashboard'
VERSION   = '1.0'
MIN_SCORE = 5          # below this = roast
STREAK_GOAL = 7        # consecutive days above MIN_SCORE
HISTORY_FILE = 'history.json'

# Session vars (filled in later)
user_name   = ''   # str
mood_score  = 0    # int 1-10
habit_score = 0.0  # float 0-1
is_weekend  = False  # bool
```

DAY 2	print() & input()

Role in the project: The Voice — every interaction with the user goes through these two functions.

Your dashboard IS a conversation. print() delivers information, input() collects it. Master their formatting now and the app feels polished instead of amateur.

Hints for Your Implementation

1. Welcome banner	print('='*50) then f-string with APP_NAME and today's date inside. Make it something you'd actually want to open every morning.
2. Morning intake	Four input() calls: name, mood word, mood score 1-10, hours of sleep. Print them back as a summary — 'Got it, Sam. Mood: tired (4/10). Sleep: 6h.'
3. Default on empty	input() returns '' if user just hits Enter. Use 'or': name = input('Name: ').strip() or 'Human'. Bulletproof in two keystrokes.
4. Pretty formatting	print('-'*40) for section dividers. '\n'.join(list) to print vertically in one call. print(f'{label:<20} {value:>6}') for aligned columns.

Starter Code Snippet (hint — not a solution)

```
def show_banner():
    print('='*50)
    print(f'  {APP_NAME}  v{VERSION}')
    print('='*50)

def collect_morning_info(greeting):
    print(f'\n{greeting}!')
    name  = input('Your name: ').strip() or 'Human'
    mood  = input('One word for your mood: ').strip()
    score = input('Mood score 1-10: ').strip()
    sleep = input('Hours of sleep last night: ').strip()
    print(f'\nGot it, {name}. Mood: {mood} | Score: {score} | Sleep: {sleep}h')
    return name, mood, score, sleep
```

DAY 3	Conditions (if / elif / else)

Role in the project: The Brain — conditions give your app personality and protect it from bad input.

A dashboard with no opinions is just a spreadsheet. Conditions decide what the app says, how it reacts, and when to roast you. if/elif/else is the engine of all of that.

Hints for Your Implementation

1. Mood tier function	mood_tier(score) returns 'excellent' for 9-10, 'good' for 7-8, 'okay' for 5-6, 'rough' for 3-4, 'disaster' for 1-2. Five outcomes, four elif branches.
2. Roast vs praise	if score < 4: roast hard. elif score < 6: gentle nudge. elif score < 8: acknowledge effort. else: full praise. At least 3 different roast strings, picked with random.choice().
3. Weekend mode	import datetime. datetime.date.today().weekday() returns 0=Mon to 6=Sun. If >= 5 it's weekend — lower the habit requirements and adjust the minimum score threshold.
4. Input validation gate	if not score.isdigit(): print error and re-ask (Day 4 loop). elif int(score) < 1 or int(score) > 10: print range error. else: proceed. Your first real guard clause.

Starter Code Snippet (hint — not a solution)

```
def mood_tier(score):
    if score >= 9:   return 'excellent'
    elif score >= 7: return 'good'
    elif score >= 5: return 'okay'
    elif score >= 3: return 'rough'
    else:            return 'disaster'

def get_reaction(daily_score):
    if daily_score >= 8:
        return 'CRUSHING IT. Keep this up.'
    elif daily_score >= 6:
        return 'Decent. Could push harder.'
    elif daily_score >= 4:
        return 'Mediocre. You know what to fix.'
    else:
        return 'Yikes. Tomorrow is a new chance.'
```

DAY 4	Loops (for / while)

Role in the project: The Engine — loops power habit check-ins, history analysis, and streak counting.

Repetition is the core of habit tracking. For each habit: ask done/skip. For each day in history: read its score. For each week: compute an average. Loops scale one piece of logic across unlimited data.

Hints for Your Implementation

1. Habit check-in loop	HABITS = ['Exercise','Read 30 min','Meditate','No junk food']. Loop: for habit in HABITS: ask y/n, store in dict. Build results as you go.
2. Retry while loop	Wrap every input() in while True: ... break. Invalid input? print error, continue. Valid? break. This pattern appears in EVERY input call.
3. History aggregation	Loop history entries: for entry in history: total += entry['daily_score']. Then avg = total / len(history). Classic accumulator pattern.
4. Streak counter	Loop history in REVERSE: for entry in reversed(history). Count consecutive days where score >= STREAK_GOAL. First day that fails -> break. Count at that point = streak.

Starter Code Snippet (hint — not a solution)

```
HABITS = ['Exercise','Read 30 min','No junk food','Meditate']

def run_habit_checkin():
    results = {}
    print('\n--- Habit Check-in ---')
    for habit in HABITS:
        while True:
            ans = input(f'{habit}? (y/n): ').lower().strip()
            if ans in ('y','n'):
                results[habit] = (ans == 'y'); break
            print('  y or n only.')
    done = sum(1 for v in results.values() if v)
    return results, done / len(HABITS)
```

DAY 5	Lists & Tuples

Role in the project: The Collections — habit lists, score history, roast banks, and all ordered sequences.

Lists hold ordered changeable data — your habit checklist, recent scores, roast messages. Tuples hold fixed data — valid mood tiers, days of the week. Knowing which to use shows real Python thinking.

Hints for Your Implementation

1. Roast message bank	ROAST_MESSAGES = ['You call that a day?', ...] with 5+ entries. random.choice(ROAST_MESSAGES) picks one. Separate PRAISE_MESSAGES for high scorers. Lists make the app feel alive.
2. Recent scores window	After loading history: recent = [e['daily_score'] for e in history[-7:]]. This list comprehension (a loop in one line) is your first Pythonic moment.
3. Tuple for fixed data	VALID_TIERS = ('excellent','good','okay','rough','disaster'). Tuple because it never changes. Check: if tier not in VALID_TIERS: raise error.
4. Sort and slice	sorted(recent) gives ascending. sorted(recent)[-1] = best. sorted(recent)[0] = worst. Or just max(recent) and min(recent). Both approaches are valid.

Starter Code Snippet (hint — not a solution)

```
ROAST_MESSAGES = [
    'That score is embarrassing. What happened?',
    'My grandma scores higher and she has dial-up.',
    'Rock bottom has a basement. You found it.',
    'One habit. Out of four. Yikes.',
]

PRAISE_MESSAGES = [
    'Absolutely locked in. This is the way.',
    'Top 1% day. Document this feeling.',
]

def get_message(score):
    import random
    if score >= 8:  return random.choice(PRAISE_MESSAGES)
    elif score < 5: return random.choice(ROAST_MESSAGES)
    return 'Solid. Keep building.'
```

DAY 6	Dictionaries

Role in the project: The Data Structure — every day's entry, habit result, and user profile is a dict.

A single day's session has many named fields: name, date, mood score, habit results, daily score. A dict holds all of these under one variable. This is structured data before you ever touch a database.

Hints for Your Implementation

1. Day entry schema	Keys: 'date','name','mood_score','mood_word','sleep','habits','daily_score','streak'. The 'habits' value is ITSELF a dict mapping habit names to True/False. Nested dicts.
2. User profile dict	PROFILE = {'name':'','goal_score':7,'streak':0,'total_sessions':0}. Load from file at startup, update each session, save back. A persistent config object.
3. Habit stats aggregation	Build: {'Exercise':{'done':12,'skipped':3}, ...} by looping history. Tells you which habits you actually keep vs which ones you lie to yourself about.
4. Dict comprehension	Convert parallel lists into a dict in one line: results = {h: r for h, r in zip(HABITS, answers)}. Dict equivalent of the list comprehension from Day 5.

Starter Code Snippet (hint — not a solution)

```
import datetime

def build_day_entry(name, mood_score, mood_word,
                    sleep, habit_results, daily_score, streak):
    return {
        'date':        str(datetime.date.today()),
        'name':        name,
        'mood_score':  mood_score,
        'mood_word':   mood_word,
        'sleep':       sleep,
        'habits':      habit_results,  # dict: habit -> bool
        'daily_score': round(daily_score, 2),
        'streak':      streak,
    }
```

DAY 7	Functions

Role in the project: The Architecture — every feature of your app should be its own named function.

Functions go from a messy pile of code to a structured, reusable, testable system. Each feature — greeting, habit check-in, score calculator, roast engine, history loader — lives in its own function, making main() readable as plain English.

Hints for Your Implementation

1. calculate_daily_score()	Parameters: mood_score (int), habit_score (float 0-1), sleep_hours (float). Weights: mood 50%, habits 40%, sleep 10% (normalise 8hrs = 1.0). Returns float 0-10. Tune the weights.
2. count_streak(history)	Takes full history list. Returns int. Loops from end. Pure function: no prints, no side effects, takes data and returns a number. Test it in isolation before integrating.
3. display_stats(history)	Prints: total sessions, average score, best score, worst score, current streak, most-skipped habit. Call it from main() after loading history.
4. main() as English	main() should read: show_banner(), name,mood,score,sleep = collect_morning_info(...), habits,habit_score = run_habit_checkin(), score = calculate_daily_score(...), save_entry(...). Pure orchestration.

Starter Code Snippet (hint — not a solution)

```
def calculate_daily_score(mood_score, habit_score, sleep_hours):
    sleep_norm = min(sleep_hours / 8.0, 1.0)
    score = (mood_score/10)*0.5 + habit_score*0.4 + sleep_norm*0.1
    return round(score * 10, 2)   # scale to 0-10

def count_streak(history, threshold=7.0):
    streak = 0
    for entry in reversed(history):
        if entry['daily_score'] >= threshold: streak += 1
        else: break
    return streak
```

DAY 8	*args, **kwargs & Lambda

Role in the project: The Power Tools — flexible functions, one-liners, and real Pythonic style.

*args handles any number of inputs. **kwargs makes function signatures configurable. Lambda collapses tiny functions into one line. Together they make your code concise, expressive, and impressive.

Hints for Your Implementation

1. *args log_event	log_event(*messages) takes any number of strings and prints them all with a timestamp. Call it with one message or five. Real *args utility.
2. **kwargs score weights	calculate_score(**weights) lets the caller pass mood=0.5, habits=0.4, sleep=0.1. Use weights.get('mood', 0.5) for defaults. Now the formula is configurable without changing the signature.
3. Lambda for sorting	history.sort(key=lambda e: e['daily_score'], reverse=True). Filter: list(filter(lambda e: e['daily_score']>=7, history)). Map: list(map(lambda e: e['daily_score'], history)).
4. Prediction lambda	predict = lambda h: round(sum(e['daily_score'] for e in h[-3:])/3,1) if h else 0. One line. Averages last 3 days. Assign to a variable and call like a function.

Starter Code Snippet (hint — not a solution)

```
def log_event(*messages):
    import datetime
    ts = datetime.datetime.now().strftime('%H:%M')
    for msg in messages: print(f'[{ts}] {msg}')

def calculate_score_v2(mood, habits, sleep, **weights):
    wm = weights.get('mood',0.5); wh = weights.get('habits',0.4); ws = weights.get('
sleep',0.1)
    return round((mood/10*wm + habits*wh + min(sleep/8,1)*ws)*10, 2)

# Lambdas
top_days = lambda h: sorted(h, key=lambda e: e['daily_score'], reverse=True)[:5]
predict  = lambda h: round(sum(e['daily_score'] for e in h[-3:])/3,1) if h else 0
```

DAY 9	Modules & Imports

Role in the project: The Toolkit — standard library modules that make your app feel like real software.

You don't build everything from scratch. Python's standard library has datetime, random, json, os, and shutil. Day 9 is where your app stops feeling like a toy and starts feeling professional.

Hints for Your Implementation

1. datetime everywhere	datetime.date.today() for the date key. datetime.datetime.now().strftime('%H:%M') for timestamps. today().weekday() to detect weekends (5=Sat, 6=Sun).
2. random for variety	random.choice(list) picks a random roast. random.randint(1,3) for random numbers. Without it your app gives the same response every time and feels robotic.
3. json as the bridge	json.dumps(data, indent=2) converts dict to pretty string. json.loads(text) converts back. The bridge between Day 6 dicts and Day 10 files.
4. os for safety	os.path.exists(HISTORY_FILE) before reading. os.path.getsize(path) to check it's not empty. Never crash because a file doesn't exist on the first run.

Starter Code Snippet (hint — not a solution)

```
import json, random, datetime, os

def get_greeting():
    h = datetime.datetime.now().hour
    if h < 12:   return 'Good morning'
    elif h < 17: return 'Good afternoon'
    else:        return 'Good evening'

def is_weekend():
    return datetime.date.today().weekday() >= 5

def safe_load_history(path):
    if not os.path.exists(path): return []
    if os.path.getsize(path) == 0: return []
    with open(path) as f: return json.load(f)
```

DAY 10	File I/O

Role in the project: The Memory — persistence turns a one-time script into an app that actually knows you.

Without File I/O your app forgets everything when it closes. With it, every session's data is stored, every streak preserved, every insight permanent. This is the difference between a calculator and a journal.

Hints for Your Implementation

1. Append-not-overwrite	history = safe_load_history(FILE). history.append(today_entry). open(FILE,'w') + json.dump(history). ALWAYS load-append-save. Never write-only or you erase everything.
2. Human-readable JSON	json.dump(..., indent=2) makes history.json readable in any text editor. You will thank yourself when debugging at 11pm.
3. Stats from file	Load history, loop, accumulate: avg = sum(e['daily_score'] for e in history) / len(history). This is the payoff for every previous day — all your data, now analysable.
4. Backup before write	import shutil; shutil.copy(HISTORY_FILE, HISTORY_FILE+'.bak') before overwriting. One extra line. Potentially saves hours of heartbreak.

Starter Code Snippet (hint — not a solution)

```
def save_entry(entry):
    history = safe_load_history(HISTORY_FILE)
    history.append(entry)
    with open(HISTORY_FILE, 'w') as f:
        json.dump(history, f, indent=2)
    print(f'Entry saved. Total sessions: {len(history)}')

def display_all_stats():
    history = safe_load_history(HISTORY_FILE)
    if not history: print('No history yet.'); return
    scores = [e['daily_score'] for e in history]
    print(f'Sessions: {len(history)} | Avg: {sum(scores)/len(scores):.1f}')
    print(f'Best: {max(scores)} | Worst: {min(scores)} | Streak: {count_streak(histo
ry)}d')
```

Putting It All Together — main() Skeleton

Every line below calls a function you've been given hints to build. Fill in the functions and this skeleton becomes a working app.

```
def main():
    show_banner()                            # Day 1, 2
    greeting = get_greeting()                # Day 9
    name, mood_word, mood_str, sleep_str = collect_morning_info(greeting)  # Day 2
    mood_score  = validate_int_input(mood_str,  1, 10)  # Day 3 + 4
    sleep_hours = validate_float_input(sleep_str, 0, 24) # Day 3 + 4
    habit_results, habit_score = run_habit_checkin()     # Day 4, 5
    history = safe_load_history(HISTORY_FILE)            # Day 9, 10
    streak  = count_streak(history)                      # Day 4, 7
    daily_score = calculate_daily_score(
        mood_score, habit_score, sleep_hours)            # Day 7, 8
    entry = build_day_entry(
        name, mood_score, mood_word, sleep_hours,
        habit_results, daily_score, streak)              # Day 6
    save_entry(entry)                                     # Day 10
    reaction = get_reaction(daily_score)                  # Day 3, 5
    tomorrow = predict(history + [entry])                 # Day 8 lambda
    print(f'  Score: {daily_score}/10 | Streak: {streak} days')
    print(f'  {reaction}')
    print(f'  Tomorrow prediction: {tomorrow}/10')
    display_all_stats()                                   # Day 4, 7, 10

if __name__ == '__main__':
    main()
```

5 Crazy Challenge Tasks (After Core Is Working)

CHALLENGE	WHAT TO BUILD	HINT
Weekly Report	Every Sunday auto-generate a 7-day summary: avg score, best day, worst day, habit completion rates.	*Check weekday()==6 in main(). Pull history[-7:]. Compute per-field averages.*
ASCII Score Chart	Print a mini bar chart of last 7 scores using only print() and loops. No matplotlib.	*bar = '#' * int(score). Print date label + bar + score. Align with f-string padding.*
Habit Accountability	Find your most-skipped habit across all history and call it out in the roast by name.	*Build habit_stats dict from history. Sort by done/total ratio. Worst habit = lowest ratio.*
Multi-User Mode	Support multiple users, each with their own history file, chosen at startup.	*HISTORY_FILE = f'{name.lower()}_history.json'. os.path.exists() per user.*
30-Day Goal Tracker	Let user set a 30-day goal at first run. Show progress: days remaining, needed average to hit target.	*Save goal in profile dict. Compute needed_avg = (goal_total - current_sum) / days_remaining.*

Remember: this document gives HINTS, not answers. The project is deliberately more ambitious than any single day's exercises — that's the point. The struggle IS the learning.